Programa:	Advanced System Optimizer 2.01.2
Protección:	Trial 30 días + DLL empakada con ASProtect
Objetivos:	Desempakar la DLL y evitar la limitación
URL descarga:	http://www.systweak.com/
Dificultad:	No es difícil
Herramientas:	OllyDbg 1.10 + PEiD 0.93 + Import Reconstructor 1.6 + ResHacker (opcional)
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinos] Fecha: 31-10-2005

Hola a todo CracksLatinoS..!!

Hoy vamos a atacar de frente un tipo de protección que yo nunca había abordado en un tute, hay uno muy similar por mi amigo [Juan Jose] (un saludo, jeje..) donde aborda una protección de este tipo, es en la teoría n°143

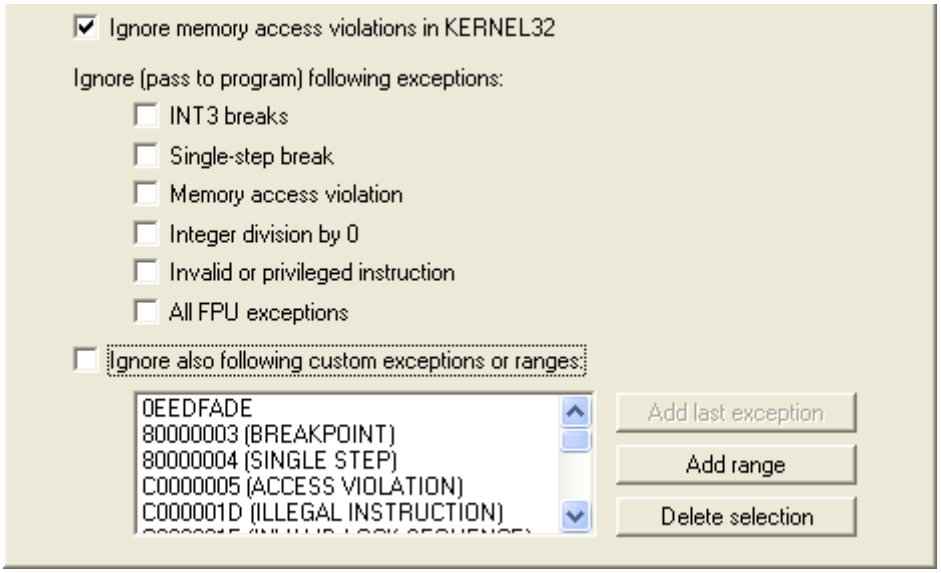
143-ASProtect en una DLL por JUAN JOSE.rar

es de obligada lectura por su calidad y porque (al menos que yo sepa) fue una de las primeras teorías donde se desempakaba una DLL. Dicho esto entramos al toro ;-)

Instalamos el programa y de momento vemos una salvajada de ejecutables, jeje.. una jauría vaya, de momento ejecutamos el primero que nos encontramos que es "adblock.exe" están ordenados alfabéticamente así que empezaremos con este, lo analizamos con PEiD y vemos que no está empaquetado ni nada:

Microsoft Visual C++ 7.0 Method2 [Debug]

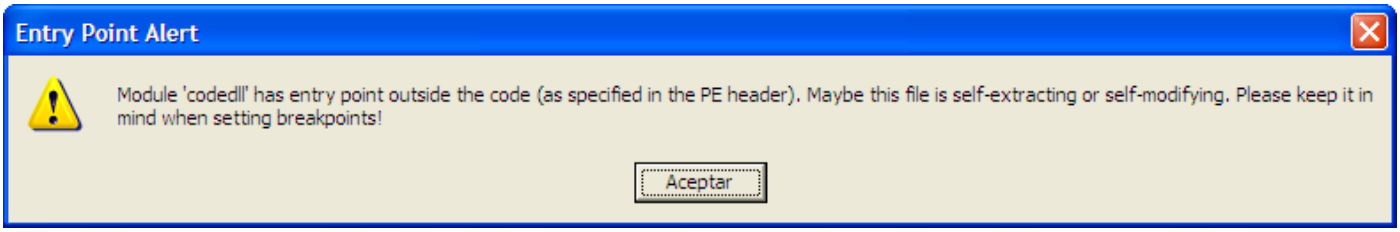
Ahí está es un "Visual C++" lo que hemos dicho no está ni empaquetado ni encriptado, esto puede parecer que nos pone las cosas fáciles pero no será así, muy difíciles tampoco serán eso lo iremos viendo sobre la marcha, de momento abrimos Olly y cargamos el ejecutable " adblock.exe" que es con el que vamos a trabajar de momento, dejamos Olly configurado para que vaya parando en todas las excepciones que se vayan produciendo, o sea de esta manera:



Así irá parando en todas las excepciones pasándonos el control de lo que hacer, ejecutamos el programa con F9 y olly para:

```
01178539   8900          MOV DWORD PTR DS:[EAX],EAX
0117853B   90            NOP
0117853C   E9 57010000   JMP 01178698
01178541   FF75 EC       PUSH DWORD PTR SS:[EBP-14]
01178544   E8 36FEFFFF   CALL 0117837F
01178549   59            POP ECX
0117854A   C3            RETN
```

típica excepción de ASProtect ¿recordáis? típico ACCESS_VIOLATION, estas las podemos ir superando con tranquilidad, además cuando cargamos el .exe en Olly nos avisa de que la librería " codedll.dll" tiene el EP fuera de la sección .text:



Esto nos da una idea de que esta librería está comprimida, de momento seguiremos superando excepciones hasta ver que pasándolas todas nos paramos en el EP del ejecutable que queremos estudiar:

```
0040F0C3  $  6A 74          PUSH 74
0040F0C5  .  68 C8144100    PUSH adblock.004114C8
0040F0CA  .  E8 F5010000    CALL adblock.0040F2C4
0040F0CF  .  33DB           XOR EBX,EBX
0040F0D1  .  895D E0        MOV DWORD PTR SS:[EBP-20],EBX
0040F0D4  .  53             PUSH EBX                                  pModule => NULL
0040F0D5  .  8B3D 2C10410   MOV EDI,DWORD PTR DS:[<&KERNEL32.GetMod   kernel32.GetModuleHandleA
0040F0DB  .  FFD7           CALL EDI                                  GetModuleHandleA
0040F0DD  .  66:8138 4D5A   CMP WORD PTR DS:[EAX],5A4D
```

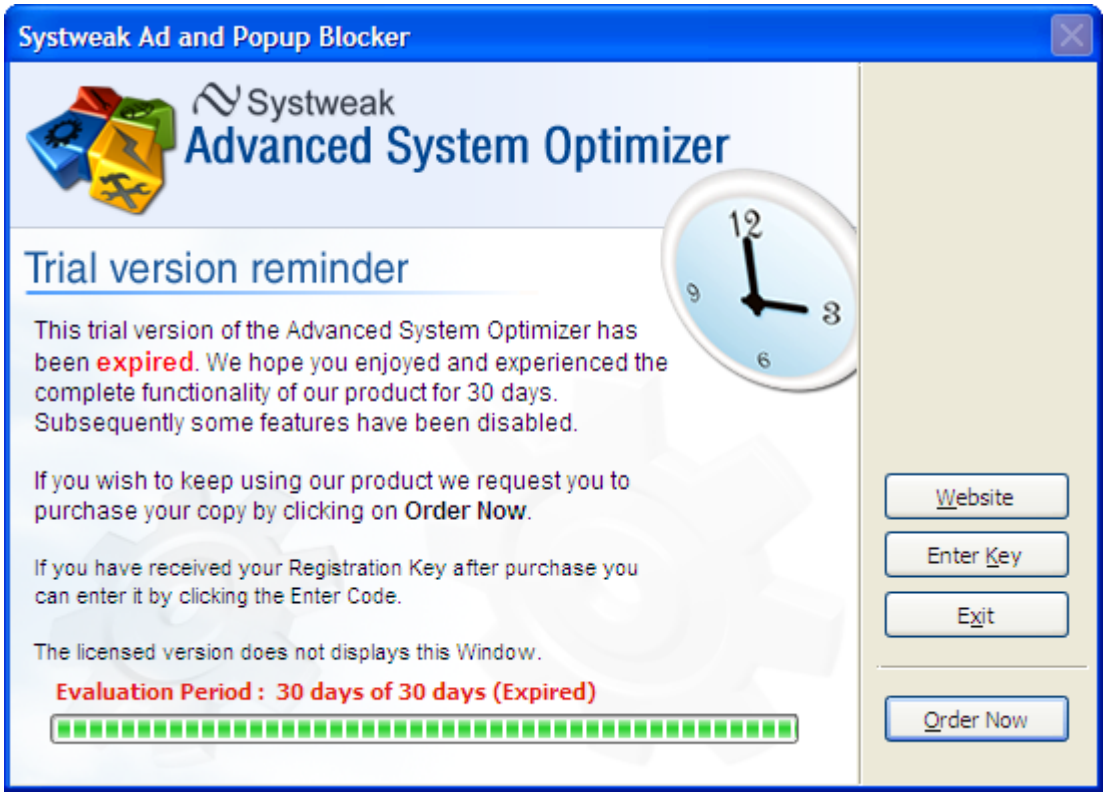
Hemos tenido que ir superando excepciones para llegar al Entry_Point ¿? esto no lo había visto nunca, veremos que más sorpresas nos deparan, la verdad que es apasionante el no saber lo que nos podemos encontrar, bueno una vez aquí ya podemos ejecutar normalmente el programa con F9 y paramos aquí:

```
01157C19   ED              IN EAX,DX                          I/O command
01157C1A   81FB 68584D56   CMP EBX,564D5868
01157C20 v 75 04           JNZ SHORT 01157C26
01157C22   C645 FF 01      MOV BYTE PTR SS:[EBP-1],1
01157C26   8A45 FF         MOV AL,BYTE PTR SS:[EBP-1]
```

Una excepción típica de acceso a puerto desde Ring3 sin privilegios, esto lo sé gracias a k0rt (otro saludo amigo) para ejecutarla deberíamos tener acceso a Ring0, miramos en la barra de estado y vemos "Privileged Instruction" que es lo que hemos dicho, esta de momento la podemos superar con Shift+F9 luego vuelve a detenerse Olly en otra excepción típica de ASProtect como las que ya hemos superado antes, las superamos todas, esta será la última:

```
01298E2D   8900            MOV DWORD PTR DS:[EAX],EAX
01298E2F   90              NOP
01298E30 v E9 57010000     JMP 01298F8C
01298E35   FF75 EC         PUSH DWORD PTR SS:[EBP-14]
01298E38   E8 42F5FFFF     CALL 0129837F
01298E3D   59              POP ECX
01298E3E   C3              RETN
```

si superamos esta con Shift+F9 nos aparece la típica NAG del programa:



Haciendo unas pruebas me expiró el programa pero en un inicio lógicamente todavía tenemos los 30 días del Trial, aunque esté expirado podemos trabajar no habrá ninguna diferencia para el que le queden días podrá seguir el tute con normalidad (lo he comprobado antes) cuando está la nag en pantalla pausamos Olly con F12 y miramos al stack que siempre nos ofrece mucha información:

```
0012EAD4  1011C688  codedll.1011C688
0012EAD8  00000000
0012EADC  0012EABC
0012EAE0  0012EF74  Pointer to next SEH record
0012EAE4  7C208FEC  SE handler
0012EAE8  00000000
0012EAEC  0012EF80
0012EAF0  10002A65  RETURN to codedll.10002A65 from codedll.10036310
0012EAF4  00415900  adblock.00415900
0012EAF8  00000000
0012EAFC  00000000
0012EB00  100459C0  codedll.100459C0
0012EB04  00000001
```

La línea resaltada es la que a nosotros nos interesa de momento, nos situamos sobre ella y pulsamos Enter para caer en la parte del código:

```
10002A52   E8 75000000    CALL codedll.10002ACC
10002A57   8D8D 80FBFFFF  LEA ECX,DWORD PTR SS:[EBP-480]
10002A5D   895D FC        MOV DWORD PTR SS:[EBP-4],EBX
10002A60   E8 AB380300    CALL codedll.10036310      <- Aquí nos muestra la nag...
10002A65   83F8 01        CMP EAX,1
10002A68 v 74 25          JE SHORT codedll.10002A8F
10002A6A   53             PUSH EBX
```

de momento ponemos un BP común en :10002A65 y pulsamos F9 para dejar correr la aplicación, si pulsamos EXIT el retorno es EAX==2 pero si nos queda Trial y pulsamos Try retorna EAX==1, ya hemos pillado el call que se encarga de mostrar la nagscreen inicial, podemos buscar un salto limpio que supere esta dirección:

```
10002A35   891E           MOV DWORD PTR DS:[ESI],EBX
10002A37 v EB 67          JMP SHORT codedll.10002AA0
10002A39   E8 C4F7FFFF    CALL codedll.10002202                    ;cuidado con este CALL...
10002A3E   84C0           TEST AL,AL
10002A40 v 74 08          JE SHORT codedll.10002A4A                ;este es un salto MALO...
10002A42   381D A8E90610  CMP BYTE PTR DS:[?b_expired@CASOLic@@2_NC],BL ;ojo a la comparacion...
10002A48 v 74 56          JE SHORT codedll.10002AA0                ;este es un salto BUENO...
10002A4A   53             PUSH EBX
10002A4B   8D85 80FBFFFF  LEA EAX,DWORD PTR SS:[EBP-480]
10002A51   50             PUSH EAX
10002A52   E8 75000000    CALL codedll.10002ACC
10002A57   8D8D 80FBFFFF  LEA ECX,DWORD PTR SS:[EBP-480]
10002A5D   895D FC        MOV DWORD PTR SS:[EBP-4],EBX
10002A60   E8 AB380300    CALL codedll.10036310                    ;aqui se muestra la NAG...
10002A65   83F8 01        CMP EAX,1
10002A68 v 74 25          JE SHORT codedll.10002A8F
10002A6A   53             PUSH EBX
```

Ahí se ven cosas interesantes primero un salto malo controlado por un CALL (lo investigaremos) después si todo va bien llega a otra comprobación, es más que sospechoso el nombre de la variable "?b_expired@CASOLic@@_NC" nos quedamos con expired, jeje.. eso seguro que tiene algo que ver con la limitación temporal, de momento reiniciamos Olly y al llegar a la zona ponemos un HBP en el CALL de :10002A39 cuando pare entramos recordando que el retorno debe ser AL==1 para que no salte en :10002A3E sino iría disparado a el call que nos muestra la nag, entremos...

```
100029A0   FF15 F8100410  CALL DWORD PTR DS:[100410F8]
100029A6   8BF0           MOV ESI,EAX
100029A8   FF15 00100410  CALL DWORD PTR DS:[10041000]
100029AE   3D B7000000    CMP EAX,0B7
100029B3 v 75 04          JNZ SHORT codedll.100029B9
100029B5   C645 F3 01     MOV BYTE PTR SS:[EBP-D],1   <- Aqui queremos llegar...
100029B9   85F6           TEST ESI,ESI
100029BB v 74 07          JE SHORT codedll.100029C4
100029BD   56             PUSH ESI
100029BE   FF15 10100410  CALL DWORD PTR DS:[10041010]
100029C4   8D4D EC        LEA ECX,DWORD PTR SS:[EBP-14]
100029C7   FF15 C4180410  CALL DWORD PTR DS:[100418C4]     mfc71.7C1771B1
100029CD   8D4D E8        LEA ECX,DWORD PTR SS:[EBP-18]
100029D0   FF15 C4180410  CALL DWORD PTR DS:[100418C4]     mfc71.7C1771B1
100029D6   8B4D F4        MOV ECX,DWORD PTR SS:[EBP-C]
100029D9   8A45 F3        MOV AL,BYTE PTR SS:[EBP-D]  <- Aqui toma AL el valor de retorno...
100029DC   5F             POP EDI
100029DD   5E             POP ESI
100029DE   5B             POP EBX
100029DF   64:890D 000000 MOV DWORD PTR FS:[0],ECX
100029E6   C9             LEAVE
100029E7   C3             RETN
```

Esta es la zona decisiva, tenemos que llegar a la instrucción donde se pone la variable [EBP-D] a 1, podemos seguir investigando el CALL anterior a ver como podemos hacer para que retorne EAX==0B7 aunque de momento no lo haremos, vamos a probar con modificar el salto lo vamos a nopear de momento para que inicialice la variable y retorne AL==1, entonces el cambio es este:

```
100029A0   FF15 F8100410   CALL DWORD PTR DS:[100410F8]
100029A6   8BF0            MOV ESI,EAX
100029A8   FF15 00100410   CALL DWORD PTR DS:[10041000]
100029AE   3D B7000000     CMP EAX,0B7
100029B3   90              NOP
100029B4   90              NOP
100029B5   C645 F3 01      MOV BYTE PTR SS:[EBP-D],1
100029B9   85F6            TEST ESI,ESI
100029BB   74 07           JE SHORT codedll.100029C4
100029BD   56              PUSH ESI
```

Seguimos traceando el CALL y efectivamente retorna AL==1 y no salta a la salida del mismo, llegamos a la otra comparación:

```
10002A42    381D A8E90610    CMP BYTE PTR DS:[?b_expired@CASOLic@@2_NC],BL
10002A48    74 56            JE SHORT codedll.10002AA0
```

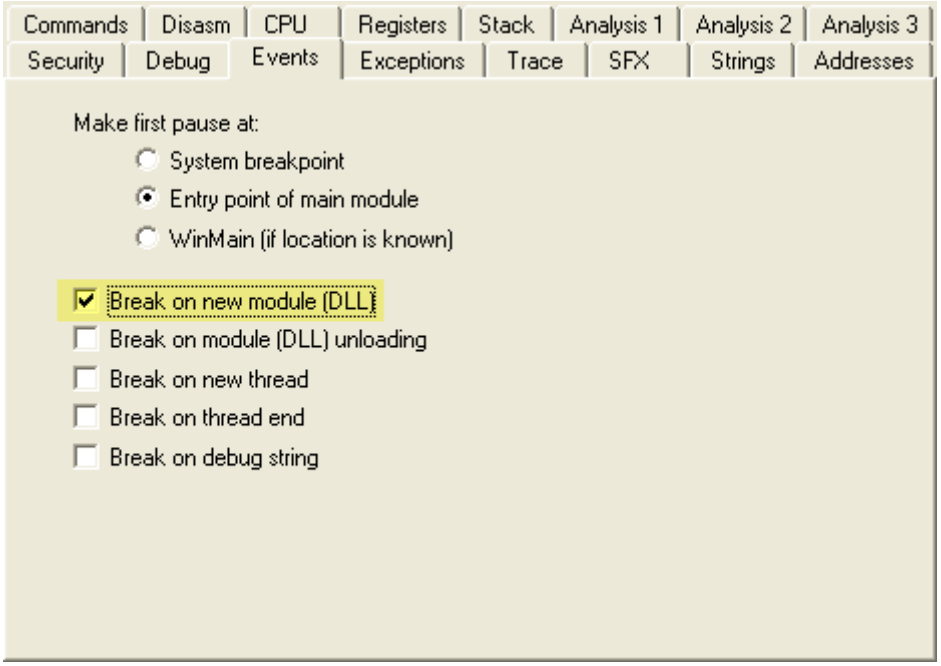
Veamos cuanto vale la variable en la ventana auxiliar:

```
BL=00
DS:[1006E9A8]=00
```

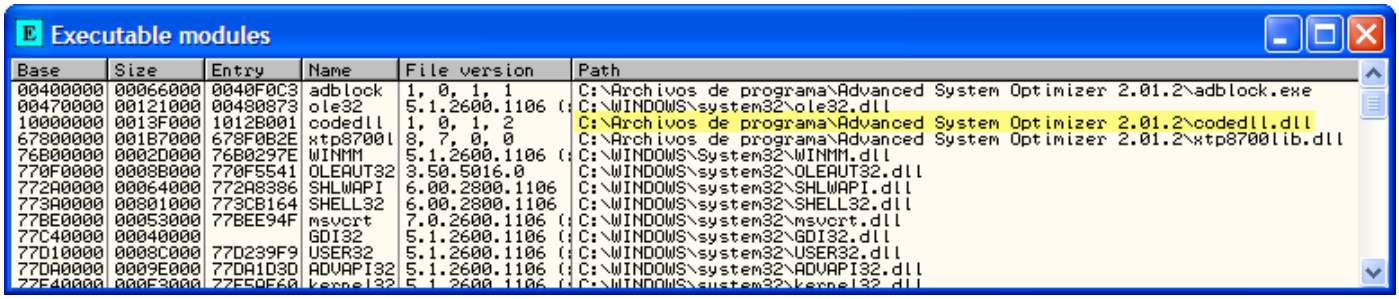
vemos que BL y [1006E9A8] valen lo mismo, se produce el salto en :10002A48 y entramos sin la nag ;-) vale vamos por el buen camino ya tenemos un punto por donde hincarle el diente, ahora tenemos un problema, veamos la dll con PEiD:

ASProtect 1.23 RC4 - 1.3.08.24 -> Alexey Solodovnikov

jeje.. nuestro amigo ASProtect una vez más se cruza en nuestro camino, esta es una versión antigua para los tiempos que corren y será una buena práctica, como nuestro amigo Juan Jose comenta en la teoría 143 lo primero que tenemos que hacer es configurar bien el Olly para que pare cuando el programa cargue una librería:



Con esto conseguimos que olly pare cuando el programa cargue cualquier librería incluida la que a nosotros nos interesa "codedll.dll" debemos reiniciar Olly claro para comenzar de nuevo, antes de tocar una tecla se detiene y queda esta ventana a la vista en primer plano:



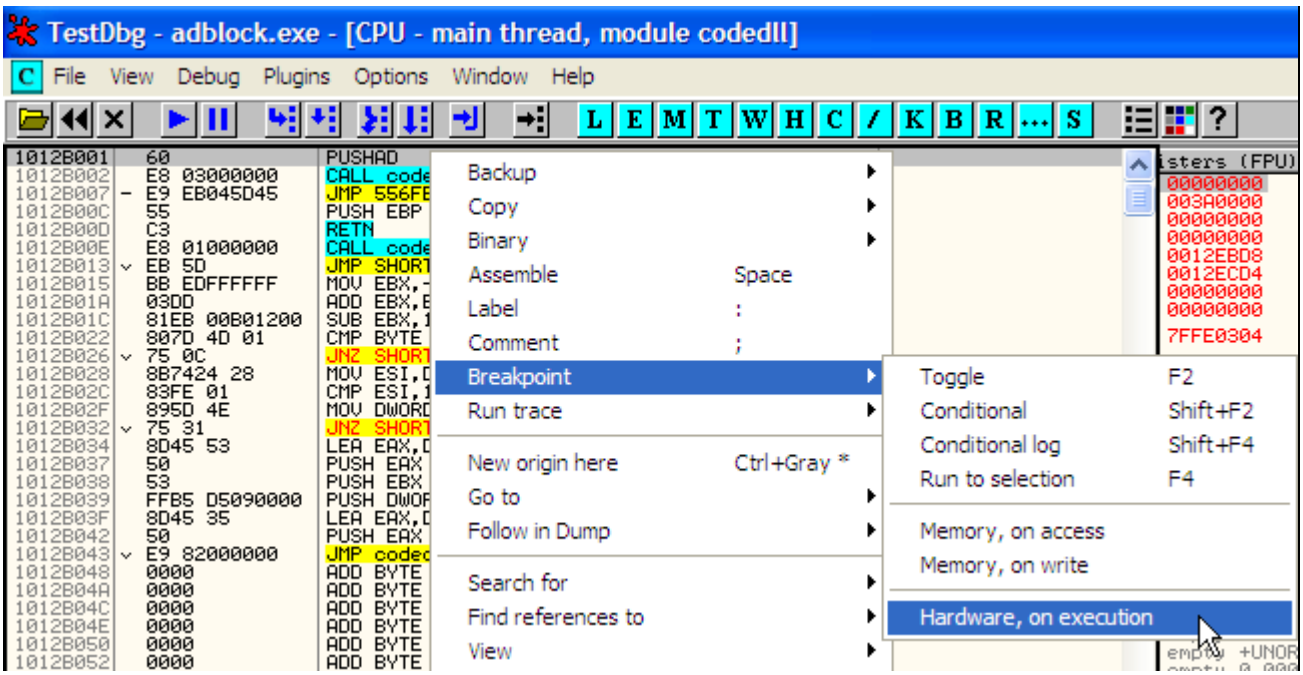
Executable modules

Base	Size	Entry	Name	File version	Path
00400000	00066000	0040F0C3	adblock	1, 0, 1, 1	C:\Archivos de programa\Advanced System Optimizer 2.01.2\adblock.exe
00470000	00121000	00480873	ole32	5.1.2600.1106 (	C:\WINDOWS\system32\ole32.dll
10000000	0013F000	1012B001	codedll	1, 0, 1, 2	C:\Archivos de programa\Advanced System Optimizer 2.01.2\codedll.dll
67800000	001B7000	678F0B2E	xtp8700l	8, 7, 0, 0	C:\Archivos de programa\Advanced System Optimizer 2.01.2\xtp8700lib.dll
76B00000	0002D000	76B0297E	WINMM	5.1.2600.1106 (	C:\WINDOWS\System32\WINMM.dll
770F0000	0008B000	770F5541	OLEAUT32	3.50.5016.0	C:\WINDOWS\system32\OLEAUT32.dll
772A0000	00064000	772A8386	SHLWAPI	6.00.2800.1106	C:\WINDOWS\system32\SHLWAPI.dll
773A0000	00801000	773CB164	SHELL32	6.00.2800.1106	C:\WINDOWS\system32\SHELL32.dll
77BE0000	00053000	77BEE94F	msvcrt	7.0.2600.1106 (	C:\WINDOWS\system32\msvcrt.dll
77C40000	00040000		GDI32	5.1.2600.1106 (	C:\WINDOWS\system32\GDI32.dll
77D10000	0008C000	77D239F9	USER32	5.1.2600.1106 (	C:\WINDOWS\system32\USER32.dll
77DA0000	0009E000	77DA1D3D	ADVAPI32	5.1.2600.1106 (	C:\WINDOWS\system32\ADVAPI32.dll
77E40000	000F3000	77E50E60	kernel32	5.1.2600.1106 (	C:\WINDOWS\system32\kernel32.dll

Selecciono la víctima ahora pulsamos con el botón derecho sobre esta línea y en el menú que aparece elegimos "Follow Entry" y caemos aquí:

```
1012B001   60              PUSHAD
1012B002   E8 03000000     CALL codedll.1012B00A
1012B007 - E9 EB045D45     JMP 556FB4F7
1012B00C   55              PUSH EBP
1012B00D   C3              RETN
1012B00E   E8 01000000     CALL codedll.1012B014
1012B013 v EB 5D           JMP SHORT codedll.1012B072
1012B015   BB EDFFFFFF     MOV EBX,-13
```

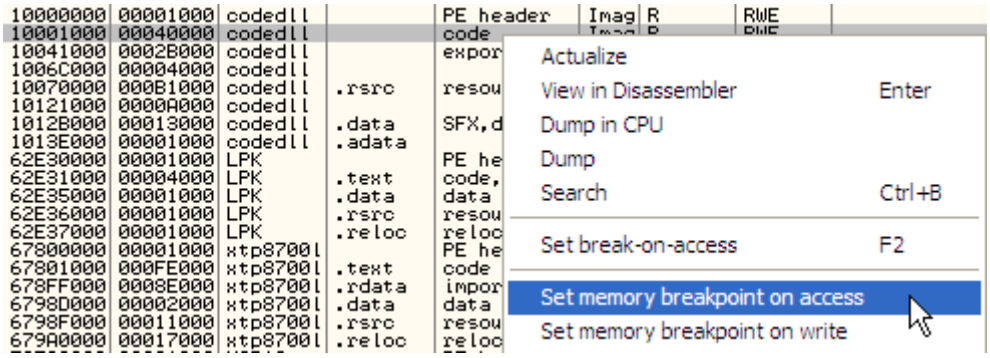
No os suena de algo este inicio? es el típico de ASProtect casi siempre lo hace así (el método del PUSHAD-POPAD no funciona eh! jeje..) esto significa que estamos en el Entry_Point de la librería en cuestión, le ponemos un HBP:



una vez puesto el BP volvemos a configurar Olly como lo teníamos para que no esté continuamente parando en cada librería cargada sería muy engorroso, el HBP nos detendrá allí así que tenemos la trampa preparada ;-)) damos RUN y efectivamente paramos allí, a partir de este momento es como si quisiéramos desempakar un ejecutable sólo cambiarán algunos detalles que iremos viendo, de momento vamos a tratar de hallar el OEP de la dll (que también tiene claro) tenemos configurado Olly para que se detenga en todas las excepciones, damos RUN y éstas se van sucediendo hasta que llega una que si la pasamos nos aparece la NAG, la anotamos:

```
00B439EC   3100            XOR DWORD PTR DS:[EAX],EAX
00B439EE   64:8F05 000000  POP DWORD PTR FS:[0]
00B439F5   58              POP EAX
00B439F6   833D B07EB400   CMP DWORD PTR DS:[B47EB0],0
00B439FD v 74 14           JE SHORT 00B43A13
00B439FF   6A 0C           PUSH 0C
```

Esta es la última, reiniciamos todo y llegamos a ella otra vez, ahora en lugar de superarla ponemos un BPM en la primera sección de la librería:



Y luego sí que la superamos con normalidad para romper en el OEP de la misma, jeje.. ya va quedando menos trabajo, lo anotamos también por su puesto:

```
10037A58   6A 0C          PUSH 0C
10037A5A   68 781E0410    PUSH codedll.10041E78
10037A5F   E8 D8000000    CALL codedll.10037B3C
10037A64   33C0           XOR EAX,EAX
10037A66   40             INC EAX
10037A67   8945 E4        MOV DWORD PTR SS:[EBP-1C],EAX
```

Bueno ya tenemos otro dato importante, ahora quedaría delimitar la iat, esto no es difícil, buscamos un CALL o un JMP que apunte a la misma y lo seguimos con "Follow in dump->Memory address" para caer en plena tabla, a partir de ahí subimos hasta encontrar el inicio que es este:

```
Address   Hex dump                                          ASCII
10041000  4C 03 B5 00 CC D9 B5 00 A4 17 B4 00 61 D9 E5 77  L♥µ.╠┘µ.ñ↨┤.a┘åw
10041010  F8 02 B5 00 D8 D9 B5 00 E4 D9 B5 00 F0 D9 B5 00  °☻µ.╪┘µ.ä┘µ.-┘µ.
10041020  00 DA B5 00 10 DA B5 00 20 DA B5 00 34 DA B5 00  .┌µ.►┌µ. ┌µ.4┌µ.
10041030  F0 A6 E5 77 44 DA B5 00 54 DA B5 00 64 DA B5 00  -ªåwD┌µ.T┌µ.d┌µ.
10041040  74 DA B5 00 80 DA B5 00 90 DA B5 00 A0 DA B5 00  t┌µ.Ç┌µ.É┌µ.á┌µ.
```

Ya lo tenemos, lo anotamos y buscamos el final de la misma:

```
Address   Hex dump                                          ASCII
10041C14  90 71 8E 67 60 71 8E 67 30 16 8A 67 10 71 8E 67  ▒qŽg`qŽg0►èg►qŽg
10041C24  30 12 8A 67 B0 13 8A 67 10 14 8A 67 F0 E1 98 67  0↨èg▒!!èg►¶èg-ßÿg
10041C34  00 B2 86 67 10 C9 87 67 10 3E 8B 67 40 4F 8B 67  .▒åg►╔çg►>ïg@Oïg
10041C44  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
10041C54  33 51 CF 40 00 00 00 00 02 00 00 00 39 00 00 00  3Q╧@....☻...9...
```

Con estos dos valores podemos obtener su longitud:

10041C44 - 10041000 = 1C44

longitud_iat: 1C44

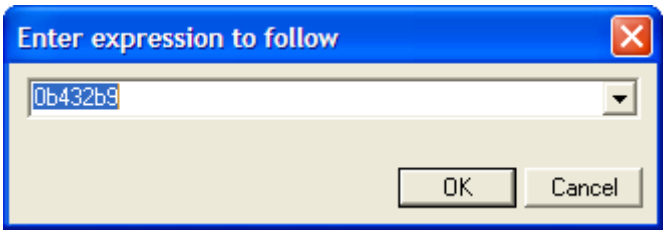
este dato lo anotamos y lo tenemos en cuenta para el Import Reconstructor que viene después, de momento vamos a buscar un puntero que esté redirigido, por ejemplo:

```
Address   Hex dump                                          ASCII
10041000  4C 03 B5 00 CC D9 B5 00 A4 17 B4 00 61 D9 E5 77  L♥µ.╠┘µ.ñ↨┤.a┘åw
10041010  F8 02 B5 00 D8 D9 B5 00 E4 D9 B5 00 F0 D9 B5 00  °☻µ.╪┘µ.ä┘µ.-┘µ.
10041020  00 DA B5 00 10 DA B5 00 20 DA B5 00 34 DA B5 00  .┌µ.►┌µ. ┌µ.4┌µ.
```

Este es el que nos va a servir de ayuda para localizar el call semi_mágico que ASProtect tiene ;-) lo que haremos será reiniciar olly y llegar a la primera excepción, tildar las excepciones momentáneamente para que Olly no pare y no se haga pesado llegar al lugar, luego poner un BPM común en la entrada mala y pulsar F9, en un momento aterrizamos en esta parte:

```
00B432AF   56             PUSH ESI
00B432B0   8B45 0C        MOV EAX,DWORD PTR SS:[EBP+C]
00B432B3   50             PUSH EAX
00B432B4   E8 47FCFFFF    CALL 00B42F00
00B432B9   E8 7EFEFFFF    CALL 00B4313C   <---  Este CALL lo nopeamos..!!
00B432BE   8B17           MOV EDX,DWORD PTR DS:[EDI]
00B432C0   8902           MOV DWORD PTR DS:[EDX],EAX
00B432C2 v EB 7E          JMP SHORT 00B43342
```

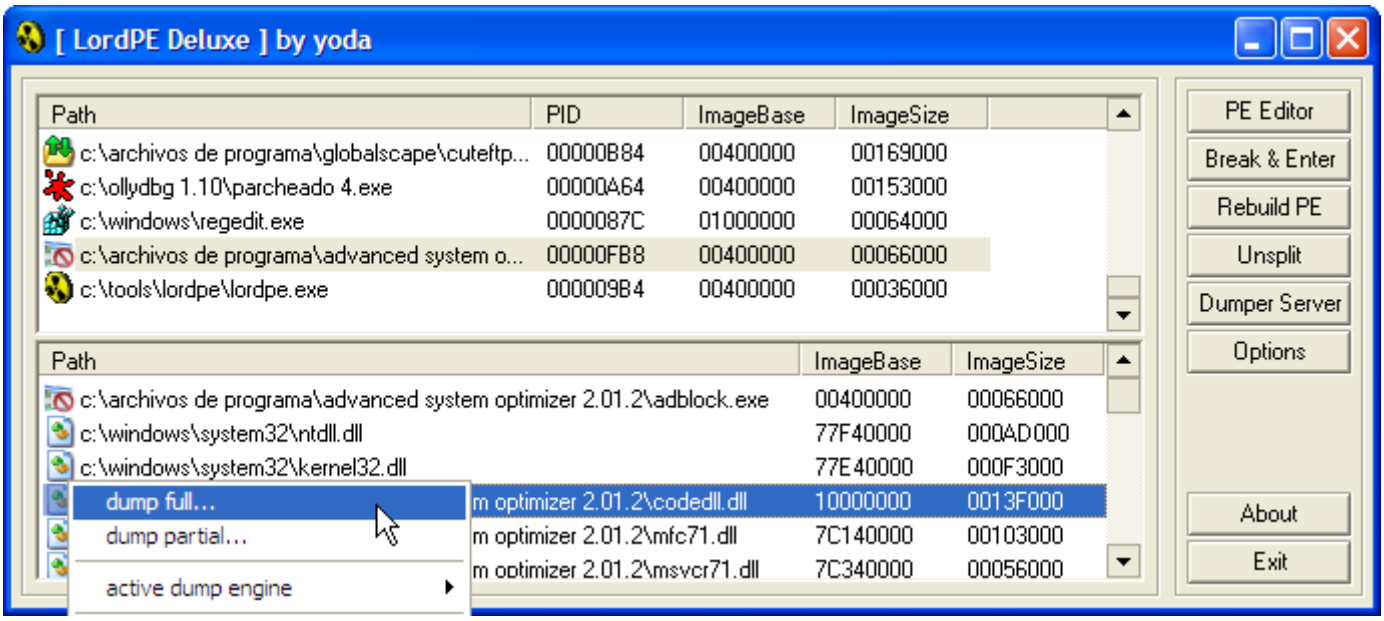
Ahí se ve en la imagen, en esta versión el call buscado es el segundo de ellos, podéis probar a poner un BP en el primero de ellos y ver que pone en EAX la api buena y tras superar el segundo cambia el valor por uno malo, de esto deducimos que el call a nopear es el segundo de ellos, pero no lo haremos ahora porque ya nos ha metido algunas entradas malas seguramente, reiniciamos Olly, destildamos las excepciones para que vuelva a parar en ellas y llegamos a la primera como hicimos antes, desde allí hacemos un GO a la dirección del CALL_mágico:



una vez allí lo nopeamos impúnemente ;-)

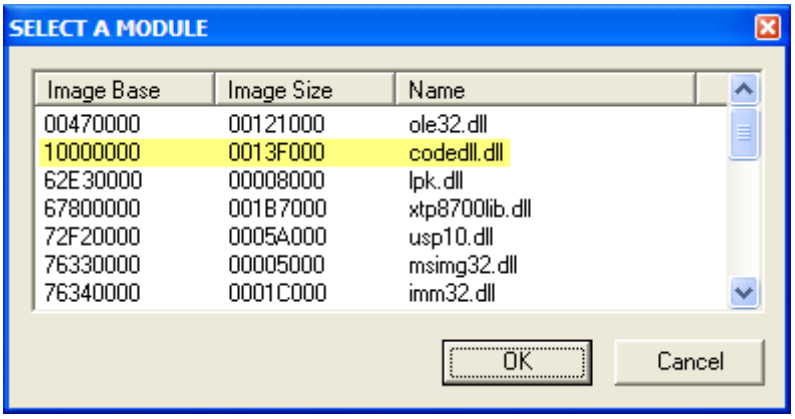
```
00B432B9   90             NOP
00B432BA   90             NOP
00B432BB   90             NOP
00B432BC   90             NOP
00B432BD   90             NOP
00B432BE   8B17           MOV EDX,DWORD PTR DS:[EDI]
```

Ahora antes de pulsar F9 ponemos un BPM en la última entrada de la iat, esto es lo que haríamos normalmente pero en este caso la última entrada que rellena está en la dirección :10041A48 así que el BPM se lo ponemos a ella, cuando pare por escritura ya tenemos la iat casi reparada, volvemos a dejar el CALL en su sitio con un "Undo Selection" esto es como siempre no cambia nada por eso no creo que haya problema, ahora volvemos a destildar las excepciones para que Olly vuelva a parar y llegamos al OEP, desde aquí dumpeamos con LordPE ya que el plugin ollydump no nos permite hacerlo, para dumpear la dll lo haremos seleccionando el proceso del ejecutable que está corriendo y en la ventanita inferior la encontramos y la dumpeamos con un "dump full"



y la guarda como dumped.dll, ya tenemos la librería en el disco pero no tiene IAT, vamos a repararla, abrimos Import Reconstructor y seleccionamos el proceso del ejecutable con el que estamos trabajando "adblock.exe" que se debe de estar ejecutando para que aparezca en la lista de procesos, cuando ya lo tenemos viene la primera diferencia ya que vamos a trabajar con una .dll y no con un .exe, lo que debemos hacer es pulsar en "pick dll" para hacerle saber que trabajaremos con un librería:

Se abrirá una pequeña ventanita donde elegimos la librería en cuestión:



Pulsamos OK y lo tenemos casi todo listo, sólo nos quedan un par de detalles más, ponemos el OEP que sabemos está en :10037A58 le restamos el imagebase :10000000 y nos queda 37A58, este valor lo ponemos en el campo OEP del Import en lugar de lo que allí haya, si pulsamos "IAT Autosearch" encontrará la iat bien luego en "Get Imports" y nos dice que quedaron 4 entradas por resolver, todos recordamos que esto siempre suele ocurrir que el CALL no repare el 100% de las entradas siempre suele dejar algunas que hay que reparar a mano, no es problema lo que vamos a hacer es una lista donde iremos poniendo los datos que vamos obteniendo de dichas entradas y como no la api correcta, comenzamos...

Primera Entrada:
IAT rva: 41008
Apunta a: B317A4
API correcta: veamos...

```
00B417A4    push ebp
00B417A5    mov ebp,esp
00B417A7    mov edx,[ebp+C]
00B417AA    mov eax,[ebp+8]
00B417AD    mov ecx,[B46484]     // DWORD value: 00B46304
00B417B3    mov ecx,[ecx]
00B417B5    cmp ecx,eax
00B417B7    jnz short 00B417C2
00B417B9    mov eax,[edx*4+B463D8]     // DWORD value: 00000000
00B417C0    jmp short 00B417C9
00B417C2    push edx
00B417C3    push eax
00B417C4    call 00B351C0     // = kernel32.dll/0191/GetProcAddress
00B417C9    pop ebp
00B417CA    retn 8
```

Esta es GetProcAddress sin duda alguna ya la podemos anotar ;-) vamos a por otra:

Segunda Entrada:
IAT rva: 41080
Apunta a: B31C64
API correcta: veamos...

```
00B41C64    push ebp
00B41C65    mov ebp,esp
00B41C67    mov eax,[ebp+8]
00B41C6A    test eax,eax
00B41C6C    jnz short 00B41C81
00B41C6E    cmp dword ptr [B47AA4],400000
00B41C78    jnz short 00B41C81
00B41C7A    mov eax,[B47AA4]     // DWORD value: 10000000
00B41C7F    jmp short 00B41C87
00B41C81    push eax
00B41C82    call 00B351B8     // = kernel32.dll/016F/GetModuleHandleA
00B41C87    pop ebp
00B41C88    retn 4
```

esta está bastante clara también es GetModuleHandleA, la anotamos y seguimos...

Tercera Entrada:
IAT rva: 41098
Apunta a: B31CC0
API correcta: veamos...

```
00B41CC0    mov eax,[B47E20]     // DWORD value: 00000ADC
00B41CC5    retn
```

Esta lo que hace es poner el valor 0ADCh en EAX es GetCurrentProcessId, sigamos...

Cuarta Entrada:
IAT rva: 410C4
Apunta a: B31C8C
API correcta: veamos...

```
00B41C8C    push dword ptr [B47E14]     // DWORD value: 0A280105
00B41C92    pop eax
00B41C93    retn
```

esta hace algo muy parecido pero el valor que deja termina en 05 que es el resultado de GetVersion si trabajamos bajo WinXP, ya las tenemos todas las reparamos y pulsamos "Fix Dump" elegimos dumped.dll que es la dll dumpeada con LordPE que hicimos antes y nos creará " dumped_.dll" esta ya tiene la iat reparada ;-) antes de utilizarla deberemos renombrarla como codedll.dll sino no la encontrará y seguirá utilizando la

empakada, la original la podemos llamar codedll_org.dll y dejarla a un lado por si la volvemos a necesitar...

Al final nos queda un cuadro-resumen de las apis reparadas como este:

IAT_rva	Apunta_a	API_válida
41008	B317A4	GetProcAddress
41080	B31C64	GetModuleHandleA
41098	B31CC0	GetCurrentProcessId
410C4	B31C8C	GetVersion

Vale parece que ya tenemos gran parte del trabajo resuelto, ejecutamos el archivo y... funciona, jeje.. la dll está bien desempakada, ahora sí podemos meternos a parchearla puesto que los bytes que veamos con Olly en memoria estarán presentes en el disco cosa que no ocurría antes al estar comprimida.

Vamos a quitarle las limitaciones, arrastramos el ejecutable "adblock.exe" a Olly para que lo cargue, ahora no nos dice que el EP de codedll.dll está fuera de la sección code, jeje.. claro ya está descomprimida antes de empezar, podemos poner BP's (a no ser que además tenga CRC), recordemos la zona en la que muestra la NAG:

```
10002A35   891E            MOV DWORD PTR DS:[ESI],EBX
10002A37 v EB 67           JMP SHORT codedll.10002AA0
10002A39   E8 C4F7FFFF     CALL codedll.10002202                    ;cuidado con este CALL...
10002A3E   84C0            TEST AL,AL
10002A40 v 74 08           JE SHORT codedll.10002A4A                ;este es un salto MALO...
10002A42   381D A8E90610   CMP BYTE PTR DS:[?b_expired@CASOLic@@2_NC],BL ;ojo a la comparacion...
10002A48 v 74 56           JE SHORT codedll.10002AA0                ;este es un salto BUENO...
10002A4A   53              PUSH EBX
10002A4B   8D85 80FBFFFF   LEA EAX,DWORD PTR SS:[EBP-480]
10002A51   50              PUSH EAX
10002A52   E8 75000000     CALL codedll.10002ACC
10002A57   8D8D 80FBFFFF   LEA ECX,DWORD PTR SS:[EBP-480]
10002A5D   895D FC         MOV DWORD PTR SS:[EBP-4],EBX
10002A60   E8 AB380300     CALL codedll.10036310                    ;aqui se muestra la NAG...
10002A65   83F8 01         CMP EAX,1
10002A68 v 74 25           JE SHORT codedll.10002A8F
10002A6A   53              PUSH EBX
```

Lo que hay que conseguir es sencillo. que no pase por :10002A65 la mejor solución siempre suele ser meternos con los CALL y no modificar simples saltos, en este caso tenemos uno con el que meternos ;-D y está en :10002A39 y debe retornar AL==1 para que no de problemas, antes lo habíamos estudiado y llegamos a la conclusión que una buena manera de conseguirlo era parcheando:

```
100029A0   FF15 F8100410   CALL DWORD PTR DS:[100410F8]
100029A6   8BF0            MOV ESI,EAX
100029A8   FF15 00100410   CALL DWORD PTR DS:[10041000]
100029AE   3D B7000000     CMP EAX,0B7
100029B3   90              NOP
100029B4   90              NOP
100029B5   C645 F3 01      MOV BYTE PTR SS:[EBP-D],1
100029B9   85F6            TEST ESI,ESI
100029BB v 74 07           JE SHORT codedll.100029C4
100029BD   56              PUSH ESI
```

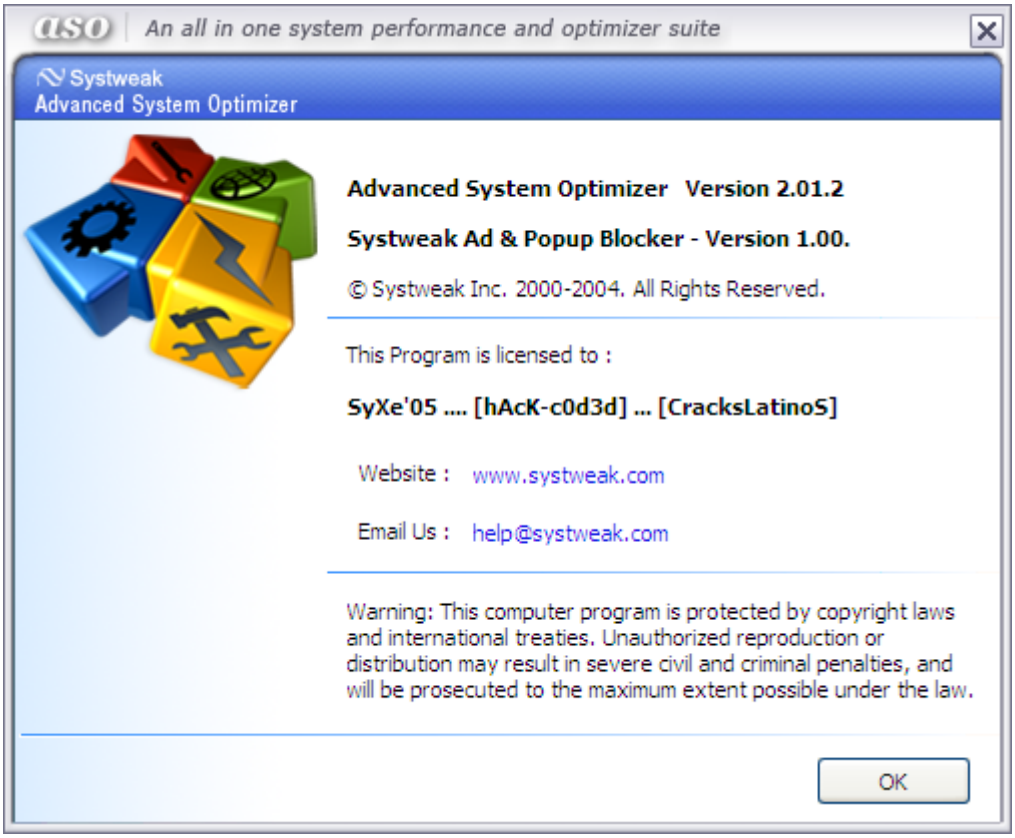
Así se llegaba a la instrucción donde se pone la variable [EBP-D] a 1 que es la que antes del RET pasa su valor a AL y es el retorno de la función, salvamos los cambios y entramos sin la nag ;-) aunque aún queda trabajo porque si adelantamos el reloj expira, la respuesta está en una imagen anterior a esta del parche donde se verifica la variable llamada "?b_expired@CASOLic@@_NC" esta almacena un resultado en función de si está o no expirado, en concreto:

variable puesta a 1 => programa EXPIRADO
variable puesta a 0 => programa NO EXPIRADO

De momento vamos a intentar ni tocar la variable porque tampoco tiene limitaciones (si las tuviese sí que la modificaríamos) como sólo tiene Trial 30 días no importa este detalle, para evitar la NAG además necesitamos saltar por encima del call de la dirección :10002A60, lo conseguimos jumpeando este salto:

```
10002A3E   84C0            TEST AL,AL
10002A40 v 74 08           JE SHORT codedll.10002A4A
10002A42   381D A8E90610   CMP BYTE PTR DS:[?b_expired@CASOLic@@2_NC],BL
10002A48 v EB 56           JMP SHORT codedll.10002AA0
10002A4A   53              PUSH EBX
10002A4B   8D85 80FBFFFF   LEA EAX,DWORD PTR SS:[EBP-480]
10002A51   50              PUSH EAX
10002A52   E8 75000000     CALL codedll.10002ACC
10002A57   8D8D 80FBFFFF   LEA ECX,DWORD PTR SS:[EBP-480]
```

Ya no la veremos más, jejeje... nos quedaría arreglar un poco la ventanita About yo lo hice con Resource Hacker y me quedó así:



Cambié el Unregistered por mi nombre y eliminé los botones "Buy Now" y "Enter Key" que no valen para nada, además en los ejecutables:

aso.exe
FileEncrypt.exe

en el menú aparecen referencias a "Buy..." que podemos eliminar parcheando ambos .exes desde ResHacker es opción personal de cada uno, yo lo he hecho está claro, quizá hubiera sido mejor seguir la pista a las variables que controlan el registro pero por esa línea llegaba a la librería " systweak.dll" empakada con "Armadillo 2.51 - 3.xx DLL Stub" y era otro curro tener que descomprimirla, etc, etc.. y todo para hacer desaparecer una estúpida cadena de texto, que no que vale más el collar que el perro, cada uno que opte por la opción que desee. Aquí hemos visto como desempakar una dll y evitar el Trial del producto, como todos los ejecutables 8que tiene muchísimos) van a esta librería a verificar el tiempo los estamos parcheando todos d emanera indirecta, el programa funka de manera ilimitada ;-)

Así que podemos decir que el trabajo está finalizado, para cualquier cosa estaré en:

syxe00@yahoo.es